HLD LLD Samjhaao

Bilkul! Ab hum SOLID Principles ke next do important principles...

Bilkul! Ab hum SOLID Principles ke next do important principles seekhenge:

1. **OCP – Open/Closed Principle**
2. **LSP – Liskov Substitution Principle**

Dono LLD (Low-Level Design) aur Java interviews mein bahut important hain.

Main dono ko real-life examples, wrong code, correct Java code, output aur interview questions ke saath samjhaunga.

Part 1: Open/Closed Principle (OCP)

1. OCP kya hai?

OCP ka full form hai **Open/Closed Principle**.

Is principle ko Bertrand Meyer ne introduce kiya tha.

Iski definition:

> *Software entities should be open for extension, but closed for modification.*

Simple Hindi mein:

- **Open for Extension:** Hum existing code mein naye features add kar sakein.
- **Closed for Modification:** Naye features add karne ke liye baar-baar existing tested code ko modify na karna pade.

Yaani existing functionality ko unnecessarily change kiye bina naye features add kar paana chahiye.

2. Real-life example samjho

Maan lo tum ek payment application bana rahe ho.

Initially tumhare application mein sirf UPI payment available hai.

Baad mein requirements aati hain:

- Credit Card payment add karo.
- Debit Card payment add karo.

- PayPal payment add karo.

Agar har baar existing payment code mein changes karne padenge, toh code complicated hota jayega.

OCP ke according hum aisa design banayenge jisme naye payment methods add karne ke liye existing payment processing logic ko modify na karna pade.

3. Java code – Without OCP (Wrong Approach)

Maan lo hum payment system bana rahe hain.

```
class PaymentService {

    public void makePayment(String type) {

        if (type.equals("UPI")) {

            System.out.println("UPI Payment");

        }
        else if (type.equals("CARD")) {

            System.out.println("Card Payment");

        }
        else if (type.equals("PAYPAL")) {

            System.out.println("PayPal Payment");

        }
    }
}
```

Abhi code sahi chal raha hai.

Lekin future mein requirement aati hai:

Net Banking payment add karo.

Toh humein existing class modify karni padegi.

```
else if (type.equals("NETBANKING")) {

    System.out.println("Net Banking Payment");

}
```

Ab problem samjho:

- Har new payment method ke liye existing class modify karni padegi.
- Code mein multiple if-else conditions badhti jayengi.
- Testing aur maintenance difficult ho jayegi.
- Existing functionality accidentally break ho sakti hai.

OCP Violation

4. Ab OCP follow karke correct code likhte hain

Is baar hum Interface ka use karenge.

Step 1: Sabse pehle ek Payment interface banayenge.

```
interface Payment {

    void pay(double amount);

}
```

Is interface ko implement karne wali har class ko `pay()` method define karna padega.

Step 2: UPI Payment class banayenge.

```
class UpiPayment implements Payment {

    @Override
    public void pay(double amount) {

        System.out.println(
            "Paid " + amount + " using UPI"
        );
```

```
    }
}
```

Step 3: Credit Card Payment class banayenge.

```
class CardPayment implements Payment {

    @Override
    public void pay(double amount) {

        System.out.println(
            "Paid " + amount + " using Card"
        );

    }
}
```

Step 4: PayPal Payment class banayenge.

```
class PaypalPayment implements Payment {

    @Override
    public void pay(double amount) {

        System.out.println(
            "Paid " + amount + " using PayPal"
        );

    }
}
```

Step 5: PaymentService banayenge.

```
class PaymentService {

    public void makePayment(
        Payment payment,
        double amount
    ) {

        payment.pay(amount);
```

```
    }
}
```

Notice karo: PaymentService ko ye nahi pata ki payment UPI se ho raha hai ya Card se.

Usko bas itna pata hai ki jo bhi object aayega, woh `Payment` interface implement karega.

5. Main class se execute karte hain

```
public class Main {

    public static void main(String[] args) {

        PaymentService service = new PaymentService();

        Payment upi = new UpiPayment();

        Payment card = new CardPayment();

        Payment paypal = new PaypalPayment();

        service.makePayment(upi, 1000);

        service.makePayment(card, 2000);

        service.makePayment(paypal, 3000);
    }
}
```

Output:

```
Paid 1000.0 using UPI
Paid 2000.0 using Card
Paid 3000.0 using PayPal
```

6. Ab future mein new payment method kaise add karenge?

Suppose interviewer bolta hai:

"Net Banking payment add karo."

Ab sirf ek new class banani hai:

```
class NetBankingPayment implements Payment {

    @Override
    public void pay(double amount) {

        System.out.println(
            "Paid " + amount + " using Net Banking"
        );

    }
}
```

Aur use aise call kar sakte hain:

```
Payment netBanking = new NetBankingPayment();

service.makePayment(netBanking, 5000);
```

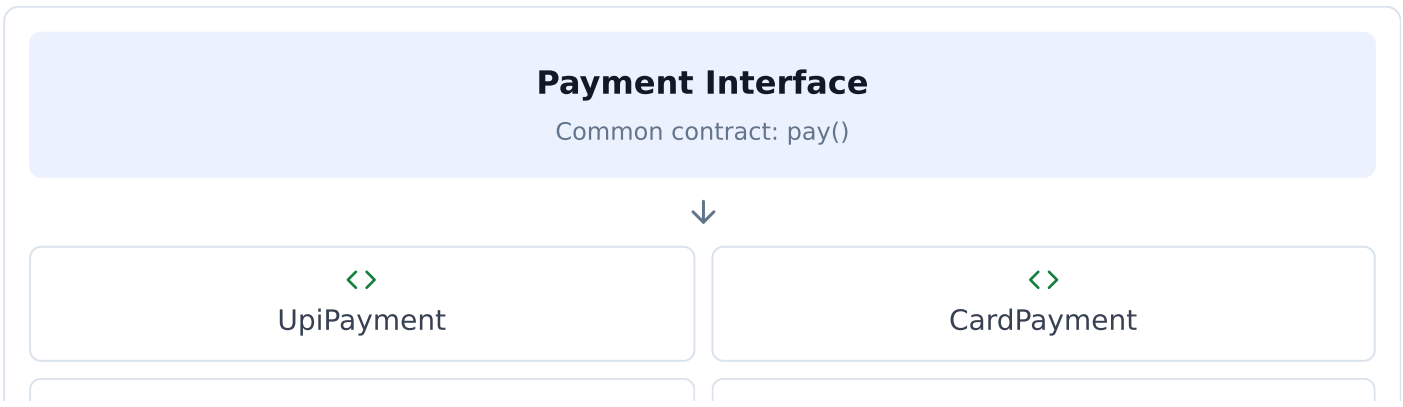
OCP Followed

Humne:

- Existing PaymentService ko modify nahi kiya.
- Existing UPI, Card aur PayPal classes ko modify nahi kiya.
- Sirf new class add ki.

Yahi Open/Closed Principle hai!

7. OCP ka diagram samjho





OCP ka main point: Existing code ko har new feature ke liye modify karne ke bajaye, naye implementations add karke functionality extend karo.

Part 2: Liskov Substitution Principle (LSP)

1. LSP kya hai?

LSP ka full form hai **Liskov Substitution Principle**.

Is principle ko Barbara Liskov ne introduce kiya tha.

Definition:

> *Objects of a superclass should be replaceable with objects of its subclasses without breaking the correctness of the program.*

Simple Hindi mein:

Agar ek parent class ka object jahan use ho raha hai, wahan uski child class ka object bhi use karne par program sahi kaam karna chahiye.

Matlab child class ko parent class ke expected behaviour ko follow karna chahiye.

Yaad rakho: Child class ko parent class ka behaviour break nahi karna chahiye.

2. Real-life example se samjho

Maan lo ek parent class hai `Bird`.

Usmein method hai:

`fly()`

Ab hum do child classes banate hain:

- Sparrow
- Penguin

Sparrow ud sakti hai.

Penguin ud nahi sakta.

Agar humne `Bird` class mein `fly()` method bana diya aur Penguin usko inherit karta hai, toh problem aa sakti hai.

Kyunki Penguin ko fly karne ke liye force kiya ja raha hai, jabki woh fly nahi kar sakta.

Ye LSP violation ka example hai.




3. Java code – Without LSP (Wrong Approach)

Sabse pehle Bird parent class banate hain.

```
class Bird {

    public void fly() {

        System.out.println("Bird is flying");

    }
}
```

Ab Sparrow class:

```
class Sparrow extends Bird {

    @Override
    public void fly() {

        System.out.println("Sparrow is flying");

    }
}
```

Ab Penguin class:

```
class Penguin extends Bird {

    @Override
    public void fly() {

        throw new UnsupportedOperationException(
            "Penguin cannot fly"
        );

    }
}
```

Ab Main class:

```
public class Main {

    public static void makeBirdFly(Bird bird) {

        bird.fly();

    }

    public static void main(String[] args) {

        Bird sparrow = new Sparrow();

        Bird penguin = new Penguin();

        makeBirdFly(sparrow);

        makeBirdFly(penguin);
    }
}
```

Output:

```
Sparrow is flying

Exception in thread "main"
```

```
UnsupportedOperationException:
Penguin cannot fly
```

Problem kya hai?

Dekho method:

```
public static void makeBirdFly(Bird bird) {

    bird.fly();

}
```

Ye method expect karta hai ki koi bhi `Bird` object aayega toh woh fly kar sakega.

Lekin Penguin object pass karne par exception aa raha hai.

Matlab child class parent class ke expected behaviour ko maintain nahi kar rahi.

LSP Violation

4. Ab LSP follow karke correct code likhte hain

Ab hum design ko improve karenge.

Hum sabhi birds ko ek common `Bird` class mein rakhenge.

Lekin `fly()` method sirf un birds ke liye banayenge jo fly kar sakte hain.

Step 1: Parent class Bird

```
class Bird {

    public void eat() {

        System.out.println("Bird is eating");

    }
}
```

Ab Bird class mein sirf common behaviour hai.

Step 2: FlyingBird interface

```
interface FlyingBird {

    void fly();

}
```

Ye interface sirf flying birds implement karenge.

Step 3: Sparrow class

```
class Sparrow extends Bird implements FlyingBird {

    @Override
    public void fly() {

        System.out.println("Sparrow is flying");

    }
}
```

Step 4: Penguin class

```
class Penguin extends Bird {

    public void swim() {

        System.out.println("Penguin is swimming");

    }
}
```

Notice karo:

Penguin ab `FlyingBird` interface implement nahi kar raha hai.

Kyunki Penguin fly nahi kar sakta.

Step 5: Main class

```
public class Main {

    public static void makeBirdEat(Bird bird) {

        bird.eat();

    }

    public static void makeBirdFly(FlyingBird bird) {

        bird.fly();

    }

    public static void main(String[] args) {

        Bird sparrow = new Sparrow();

        Bird penguin = new Penguin();

        makeBirdEat(sparrow);

        makeBirdEat(penguin);

        makeBirdFly((FlyingBird) sparrow);
    }
}
```

Output:

```
Bird is eating
Bird is eating
Sparrow is flying
```

Ab dono birds `Bird` ke roop mein use ho sakte hain aur koi unexpected exception nahi aata.

Sirf flying birds ko `makeBirdFly()` method mein pass kar sakte hain.

Yahi LSP follow karna hai.

---

5. Ek aur important example – Rectangle and Square

Ye LSP ka bahut popular interview example hai.

Maan lo humare paas Rectangle class hai.

Rectangle mein:

- Width alag set kar sakte hain.
- Height alag set kar sakte hain.

```
class Rectangle {

    protected int width;
    protected int height;

    public void setWidth(int width) {
        this.width = width;
    }

    public void setHeight(int height) {
        this.height = height;
    }

    public int getArea() {
        return width * height;
    }
}
```

Ab Square class Rectangle ko inherit karti hai.

```
class Square extends Rectangle {

    @Override
    public void setWidth(int width) {

        this.width = width;
        this.height = width;

    }
```

```
    @Override
    public void setHeight(int height) {

        this.width = height;
        this.height = height;

    }
}
```

Ab problem dekhte hain.

```
public class Main {

    public static void main(String[] args) {

        Rectangle rectangle = new Square();

        rectangle.setWidth(10);

        rectangle.setHeight(5);

        System.out.println(rectangle.getArea());

    }
}
```

Expected output agar normal Rectangle hota:

```
50
```

Lekin actual output:

```
25
```

Kyun?

Jab `setHeight(5)` call hua, Square ne width ko bhi 5 kar diya.

Isliye area 25 aa gaya.

Parent Rectangle ka expected behaviour break ho gaya.

LSP Violation

Is problem ko kaise solve karenge?

Rectangle aur Square ko forcefully inheritance mein rakhne ke bajaye ek common abstraction use kar sakte hain.

```
interface Shape {

    int getArea();

}
```

Rectangle:

```
class Rectangle implements Shape {

    private int width;
    private int height;

    Rectangle(int width, int height) {

        this.width = width;
        this.height = height;

    }

    public int getArea() {

        return width * height;

    }
}
```

Square:

```
class Square implements Shape {

    private int side;

    Square(int side) {
```

```
        this.side = side;
    }
    public int getArea() {
        return side * side;
    }
}
```

Ab dono Shape hain aur apna area correctly calculate karte hain.

Is design mein inheritance ka incorrect relationship nahi hai.

---

6. OCP vs LSP – Difference

OCP	LSP
Open/Closed Principle	Liskov Substitution Principle
New features easily add karna	Child class parent ko correctly replace kare
Existing code ko unnecessary modify na karna	Parent ka expected behaviour break na karna
Interfaces and abstraction help karte hain	Correct inheritance and contracts important hain
Example: Payment System	Example: Bird and Penguin

7. Interview ke important questions

Q1. OCP ka main purpose kya hai?

Answer: Existing code ko repeatedly modify kiye bina new functionality add karna.

Q2. OCP achieve karne ke liye kya use karte hain?

Answer: Interfaces, abstraction, polymorphism aur suitable design patterns.

Q3. LSP kisne introduce kiya?

Answer: Barbara Liskov.

Q4. Kya har child class parent class ko inherit kar sakti hai?

Answer: Technically Java mein inherit kar sakti hai, lekin inheritance tabhi use karni chahiye jab child class parent ke expected behaviour ko maintain kar sake.

Q5. Penguin example mein LSP violate kyun hua?

Answer: Kyunki parent Bird class ka `fly()` method Penguin ke liye valid nahi tha aur exception throw ho raha tha.

---

8. Ab khud test karo – Interview Quiz

SOLID Interview Practice 0/4 answered

Q1. Payment system mein new UPI ya PayPal class add karte waqt PaymentService ko change nahi karna padta. Ye kaunsa principle hai?

- SRP
- OCP
- LSP

Q2. Agar child class parent class ke method ko override karke unexpected exception throw kare, toh kaunsa principle violate ho sakta hai?

- OCP
- SRP
- LSP

Q3. Penguin ko FlyingBird interface implement karna chahiye?

- Yes
- No

Q4. OCP mein new payment method add karne ke liye kya karenge?

- PaymentService ko baar-baar modify karenge
- New Payment implementation class banayenge
- Saare old classes delete karenge

Submit Answers

9. Dono principles ko ek line mein yaad rakho

- **SRP:** Ek class ki ek responsibility honi chahiye.
- **OCP:** New functionality add karo bina existing stable code ko unnecessarily modify kiye.
- **LSP:** Child class ko parent class ki jagah use karne par program ka behaviour break nahi hona chahiye.

Ab next learning step: SOLID ke remaining do principles hain:

- ISP – Interface Segregation Principle
- DIP – Dependency Inversion Principle

In dono ko bhi isi tarah Java ke real-world examples aur wrong vs correct code ke saath samajhna tumhari LLD preparation complete karne mein help karega.

gpt-6